

Version 2.0

January 2026



OBJECT-ORIENTED PROGRAMMING

MODULE 6 - GUI, JAVA SWING, JAVA FX

Author:

Ir. Galih Wasis Wicaksono, S.Kom., M.Cs.

Yossua Agung Budianto

Muhammad Budi Kusuma

INFORMATICS LABORATORY

University of Muhammadiyah Malang

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	4
OBJECTIVES.....	4
MODULE TARGETS.....	4
PREPARATION.....	4
KEYWORDS.....	4
THEORY.....	5
1. Graphical User Interface (GUI).....	5
1.1 Why Should I Understand This?.....	5
1.2 Formal Definition.....	5
1.3 Concepts Elaboration.....	6
1.3.1 CLI vs GUI.....	6
1.3.2 Event-Driven Programming (EDP).....	7
2. Swing.....	8
2.1 Why Should I Understand This?.....	8
2.2 Formal Definition.....	8
2.3 Concepts Elaboration.....	9
2.3.1 JFrame.....	9
2.3.2 JPanel.....	11
2.3.3 Layout.....	14
BorderLayout.....	14
FlowLayout.....	15
GridLayout.....	16
2.3.4 Basic Component.....	18
JLabel.....	18
JButton.....	19
JTextField.....	21
JOptionPane.....	22
JTable.....	23
2.3.5 Event Handling in Swing.....	25
How to Make Event Handling.....	25
3. JavaFX.....	29
3.1 Why Should I Understand This?.....	29
3.2 Formal Definition.....	29
3.3 Concepts Elaboration.....	30
3.3.1 Instalation.....	30
3.3.1 Stage.....	33
Container Hierarchy in JavaFX (Stage-Scene-Node).....	34
3.3.2 Scene.....	35



3.3.3 Layout.....	37
HBox & VBox.....	37
GridPane.....	40
BorderPane.....	41
AnchorPane.....	42
StackPane.....	43
3.3.4 Components.....	45
Label.....	45
Button.....	46
TextField.....	48
ListView & TableView.....	49
3.3.5 Event Handling.....	50
How to Make Event Handling.....	51
EXERCISE.....	54
ASSIGNMENT.....	56
ASSESSMENT.....	58
Percentage Assessment.....	58



INTRODUCTION

OBJECTIVES

1. Understand the difference between a Command Line Interface (CLI) and a Graphical User Interface (GUI), as well as the importance of visual elements in bridging human-machine communication.
2. Recognize the Event-Driven Programming (EDP) paradigm, where the program flow is determined by user interactions rather than simple linear execution.
3. Understand the fundamental architecture of building desktop applications using the Java Swing library (consistent across platforms) and the modern JavaFX.

MODULE TARGETS

1. Students are able to build application windows using JFrame (Swing) and Stage (JavaFX) as the primary interface foundations.
2. Students are able to organize component layouts neatly and responsively using various Layout Managers such as BorderLayout, GridLayout, HBox, and VBox.
3. Students are able to integrate interactive components like Buttons, TextFields, Labels, and Tables to receive input and display data.
4. Students are able to implement Event Handling using Anonymous Classes or Lambda Expressions to enable functionality for buttons and other components.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson

رَضِيتُ بِاللّٰهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop or personal computer
3. IntelliJ IDE/Any Java Compiler (Eclipse VSCode)
4. "Full Poweeeeerrrrrrrr"

KEYWORDS

Graphical User Interface (GUI), CLI, Event-Driven Programming (EDP), Event Source, Event Object, Event Listener, Swing, AWT, Platform-Independent, JFrame, JPanel, Layout Manager, BorderLayout, FlowLayout, GridLayout, JLabel, JButton, JTextField, JOptionPane, JTable, JavaFX, Stage, Scene, Node, Scene Graph, HBox, VBox, GridPane, BorderPane, AnchorPane, StackPane, FXML, CSS Styling, Lambda Expression.



THEORY

1. Graphical User Interface (GUI)

1.1 Why Should I Understand This?

Up until now, you've probably been used to dealing with programs through those black-and-white command lines we call the **Command Line Interface (CLI)**. Just to add two numbers, you have to type them in, hit enter, and watch the result pop up as plain text... so boring, right?! For us developers, sure, it feels efficient. But try to imagine if that app was used by regular folks, like your grandma or a little kid. It would literally hurt their eyes just looking at it.

This is where the **Graphical User Interface**, or **GUI**, comes. The main problem GUI solves is that awkward "communication gap" between humans and machines. Humans are naturally visual creatures, let's be real, we'd much rather look at beautiful things than memorize syntax and walls of text. Without a GUI, users are often terrified of making mistakes because there are no clear visual clues about what they can or can't do. GUI flips that "scary" experience into a comfortable interaction using elements that feel familiar to the eye.

In the world of Java, being able to build a GUI basically means you're giving a **face** to all that logic you wrote. Your program isn't just invisible code running in the background anymore, it becomes a real product that anyone can touch, click, and actually find useful.

1.2 Formal Definition

Graphical User Interface (GUI) is a system of interactive visual components that computer software uses to present information and accept user input. GUIs leverage graphical representations (icons, windows, menus, and pointers) to facilitate human-computer interaction efficiently.

In the Java ecosystem, GUIs are typically built using the **Event-Driven Programming** paradigm. This means the program's flow isn't dictated linearly from the first line of code to the last anymore. Instead, it is driven by user interactions or "events," such as mouse clicks, key presses, or cursor movements.

Structurally, GUIs often follow the **WIMP** principle, which is basically the industry standard:

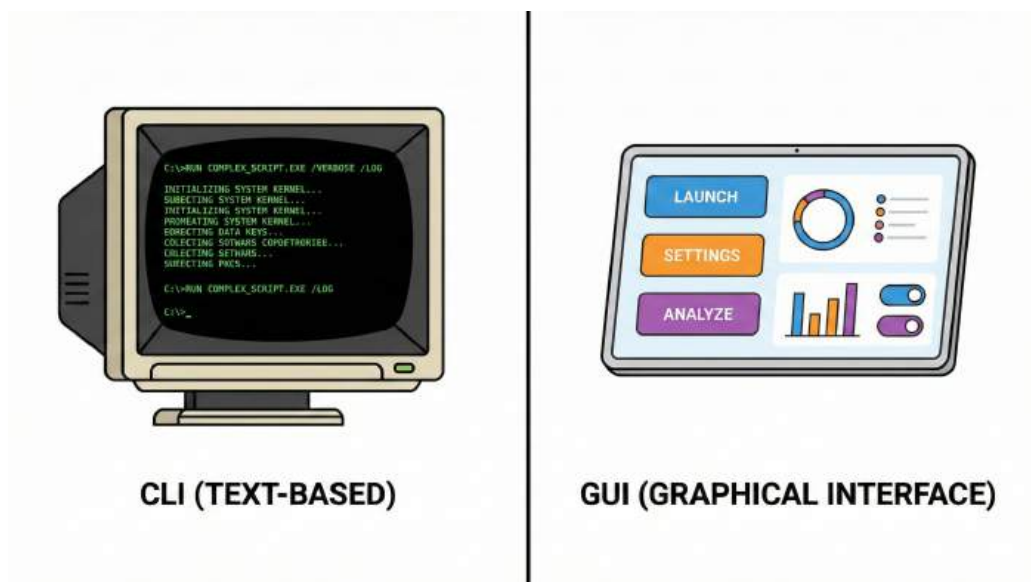
- **Windows:** The main visual areas that hold the application's content.
- **Icons:** Graphical symbols representing instructions or objects.
- **Menus:** Lists of command options that the user can execute.
- **Pointer:** An on-screen symbol (usually an arrow) representing the movement of the user's physical input device.



In Java specifically, GUI implementation is supported by standard libraries like the **Abstract Window Toolkit (AWT)**, **Swing**, and the more modern **JavaFX**. These three libraries provide the necessary building blocks (components) to construct interfaces that are complex yet organized.

1.3 Concepts Elaboration

1.3.1 CLI vs GUI



In the CLI world, the program runs the show. It asks "Enter name:", and then it just sits there waiting until you give it some input before it moves on to the next step. This pattern is really linear and rigid.

But in a GUI, you are the one in control. A GUI program usually just waits faithfully in the background, letting you decide whether you want to type, click a help button, or just close the window immediately. You can even do multiple actions at once!!! you don't have to go step-by-step like in a CLI.

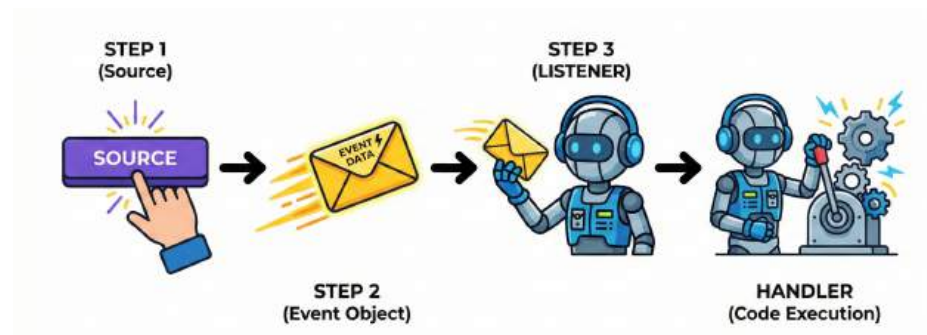
There are a few fundamental differences we need to highlight in this comparison:

- **Flow Control:** CLI is Program-Driven (the program dictates the steps), whereas GUI is User-Driven (the user decides the next action).
- **State:** GUI is stateful, meaning the application must always be aware of its current condition (for example: should the "Send" button be enabled or disabled based on the content of the text box?).
- **Interactivity:** CLI tends to be one-way and sequential, while GUI supports non-linear interaction, allowing users to perform various tasks without a forced order.



CLI	GUI
Have to type commands to use the program	Just click around to use the program
Fast for advanced users	Easy for beginner user
Examples: Terminal, CMD	Examples: WhatsApp, Chrome, YouTube

1.3.2 Event-Driven Programming (EDP)



Event-Driven Programming (EDP) is simply a paradigm in which the program flow is controlled by user actions. It doesn't just run linearly from top to bottom and then finish, instead, it sits there waiting for a specific **event** to happen before it kicks into action. If you're building desktop apps with Swing or JavaFX, you really need to get this concept down. Basically, the app's flow is decided by the user, whether it's a mouse click, typing on the keyboard, or even input from a sensor. This is exactly why we can do multiple things at once using a GUI!

Before diving deep into GUI, you gotta know these three concepts:

- **Event Source (The Trigger):** This is the component that kicks off the action, like a Button, a TextField, or a Menu.
- **Event Object (The Messenger):** As soon as a button is clicked, Java creates an object filled with all the info about that event (what was clicked, what time it happened, etc.).
- **Event Listener (The Responder):** This is the chunk of code that keeps its ear open for action. As soon as an event pops up, this listener executes the command, like showing a "Hello World!" message.

So, once you understand that GUIs run on these three concepts, next you'll learn how to actually implement them in Java Swing and JavaFX.



2. Swing

2.1 Why Should I Understand This?

When talking about Java Swing, we need to step back a bit into its history. Back then, there was something called AWT (Abstract Window Toolkit). AWT was like a translator that heavily relied on assistants in each operating system. For example, when you created a button in Windows using AWT, it would actually borrow the default button style from Windows itself, and the same applied to Mac or Linux. The problem was that every operating system had different GUI component styles, so if you built a GUI-based program in Windows, chances are it wouldn't run properly on Mac or Linux.

In short, the issue caused by the AWT framework was that its GUI component styles weren't standardized across operating systems. AWT also depended heavily on whatever components existed in your operating system. If one of the operating systems you used didn't have an "advanced table" feature, then AWT couldn't provide it for you either. We were stuck with only the most basic features because we had to follow the limitations of the weakest operating system.

Because of the problems faced by AWT, it eventually became the starting point for the birth of Swing. Unlike AWT, Swing doesn't borrow the operating system's built-in components to create a GUI, instead, Swing draws each component itself, pixel by pixel. That's why Swing can provide GUI components that look consistent across all operating systems.

2.2 Formal Definition

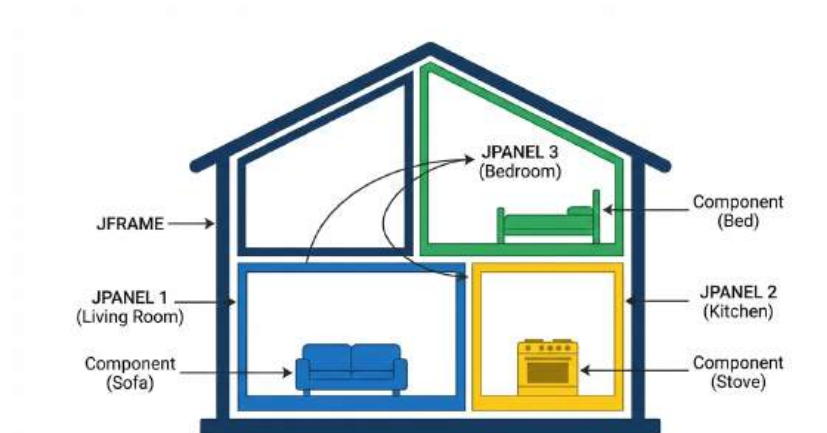
Java Swing is a library (part of the **Java Foundation Classes** or JFC) that we use to build GUIs for Java apps. It's built on top of the old AWT foundation, but it takes things to the next level by offering components that are "lightweight" and platform-independent.

Unlike its predecessor (AWT), Swing doesn't rely on the operating system's native code to render visuals. Instead, it draws everything using pure Java. Architecture-wise, it follows a modified Model-View-Controller (MVC) pattern, which nicely separates the data (Model), the visuals (View), and the logic (Controller) for each component. Plus, it's packed with a massive inventory of components, from basic stuff like buttons (`JButton`) all the way to complex heavy hitters like data tables (`JTable`) and trees (`JTree`).



2.3 Concepts Elaboration

2.3.1 JFrame



The first step in creating a GUI application with Swing is to prepare a container or window where the application will appear. In Swing, this container is called a `JFrame`. Without `JFrame`, the software application you're using right now wouldn't display its interface. Just imagine `JFrame` as the main container that holds all the GUI components.

Just imagine `JFrame` as a house. Without the house, there wouldn't be any rooms, and we wouldn't even know where to put the furniture, cuz the house itself doesn't exist, awokawokawok.

Here's how to declare the object:

```
JFrame frame = new JFrame();
```



Here is an example of its implementation:

```
import javax.swing.JFrame;
import java.awt.FlowLayout;

public class ExampleFrame {
    public static void main(String[] args) {
        // Create the JFrame object
        JFrame frame = new JFrame();

        // Set the window title
        frame.setTitle("My First App");

        // Set the window size (width 400, height 300)
        frame.setSize(400, 300);

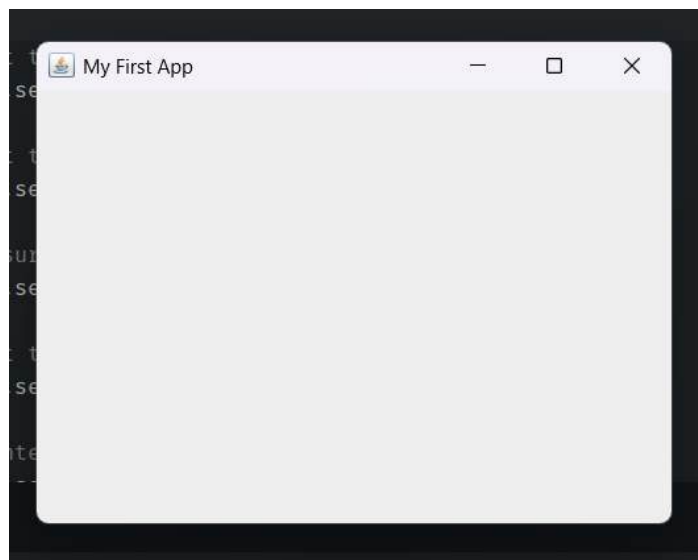
        // Ensure the program exits when the window is closed
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Set the layout so components align in a row
        frame.setLayout(new FlowLayout());

        // Center the window on the screen
        frame.setLocationRelativeTo(null);

        // Show the window on the screen (Must be at the end)
        frame.setVisible(true);
    }
}
```

Output:



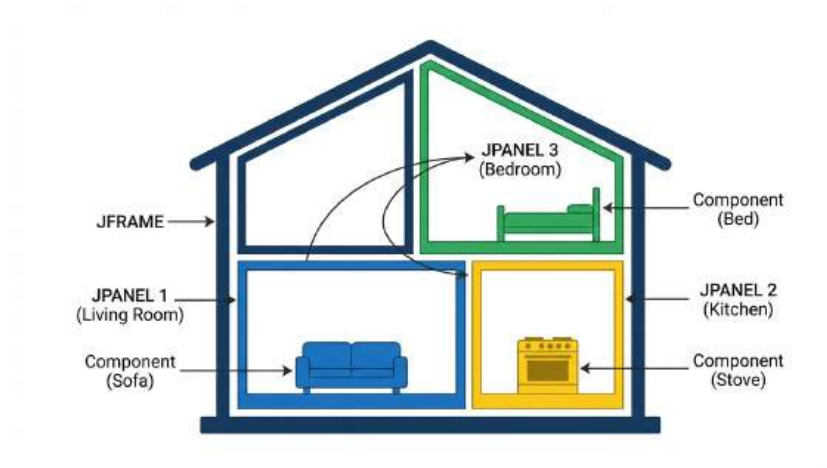
See? We've just created a Window. Well, it's still empty for now, but we'll fill it with GUI components one by one later.



There are still many `JFrame` methods you can explore through the following documentation, go ahead:

Documentation: [JFrame \(Java SE 21 & JDK 21\)](#)

2.3.2 JPanel



If `JFrame` is like a house, then `JPanel` is its rooms. The `JPanel` is used to group your GUI elements together. You might be asking, "*Why not just put buttons directly onto the JFrame?*" "The answer is... because `JPanel` is used to group components. With `JPanel`, you can divide components into several groups positioned differently within a `JFrame`.

When talking about `JPanel`, it has some interesting facts:

- Compared to `JFrame`, `JPanel` is very lightweight. We can create hundreds of `JPanel` inside a single `JFrame` without excessively burdening the computer's memory.
- One of the main strengths of `JPanel` is its ability to be **nested**. You can actually place one `JPanel` inside another.
- `JPanel` is invisible to humans like us, it exists as an unseen container to group components together.

Here's how to declare the object:

```
JPanel panel = new JPanel();
```



Example 1:

```
import javax.swing.*;
import java.awt.*;
import javax.swing.border.LineBorder;

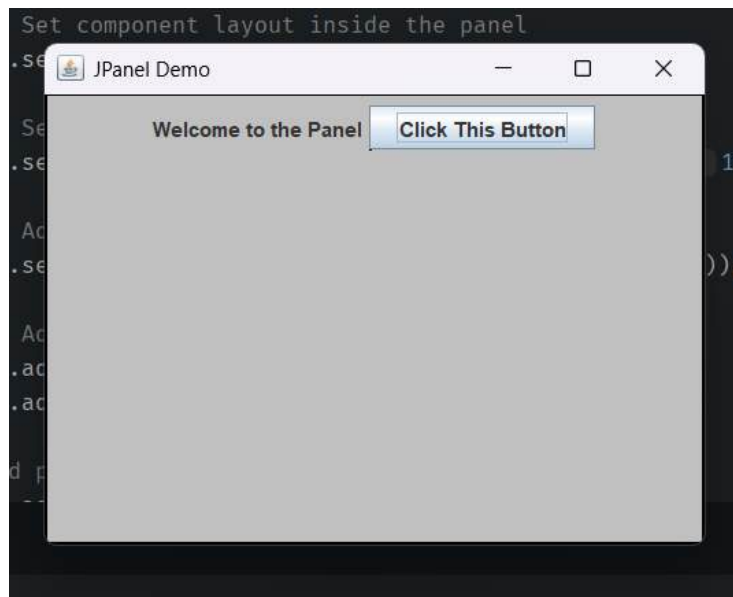
public class BasicPanelExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JPanel Demo");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 300);

        JPanel panel = new JPanel();
        panel.setBackground(Color.LIGHT_GRAY);
        panel.setLayout(new FlowLayout());
        panel.setPreferredSize(new Dimension(300, 150));
        panel.setBorder(new LineBorder(Color.BLACK, 2));

        panel.add(new JLabel("Welcome to the Panel"));
        panel.add(new JButton("Click This Button"));

        frame.add(panel);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Output 1:



Example 2:

```
import javax.swing.*;
import java.awt.*;

public class NestedPanelExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Nested Panels App");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(450, 350);
        // Using BorderLayout on the main Frame
        frame.setLayout(new BorderLayout());

        // 1. Create Header Panel (Top Section)
        JPanel headerPanel = new JPanel();
        headerPanel.setBackground(Color.DARK_GRAY);
        headerPanel.add(new JLabel("<html><font color='white'>APPLICATION TITLE</font></html>"));

        // 2. Create Content Panel (Center Section)
        JPanel contentPanel = new JPanel();
        contentPanel.setLayout(new GridLayout(2, 1, 10, 10)); // 2-row Grid
        contentPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
        contentPanel.add(new JTextField("Enter Your Name..."));
        contentPanel.add(new JTextField("Enter Email..."));

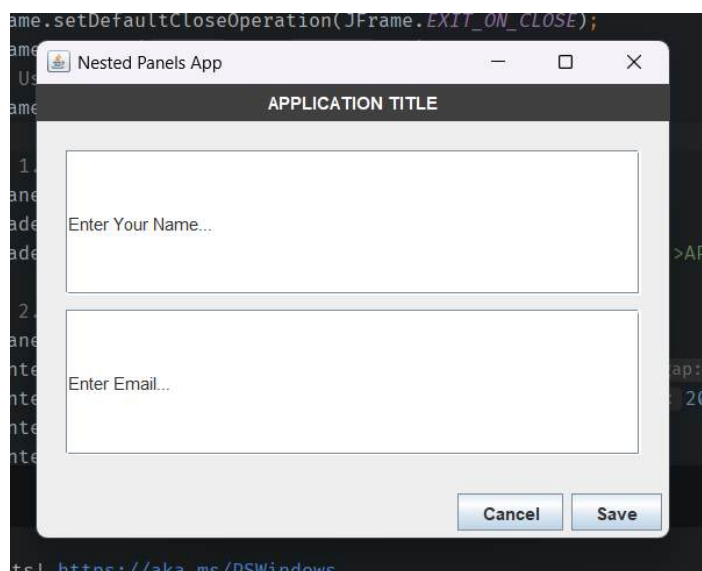
        // 3. Create Action Panel (Bottom Section)
        JPanel footerPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        footerPanel.add(new JButton("Cancel"));
        footerPanel.add(new JButton("Save"));

        // Insert small panels into different positions on the main Frame
        frame.add(headerPanel, BorderLayout.NORTH);
        frame.add(contentPanel, BorderLayout.CENTER);
        frame.add(footerPanel, BorderLayout.SOUTH);

        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```



Output 2:



Okay, maybe that's enough examples for now. For the rest, feel free to experiment and try things out to understand better. There are still many `JPanel` methods you can explore through the following documentation, go ahead:

Documentation: [JPanel \(Java SE 21 & JDK 21\)](#)

2.3.3 Layout

One of the biggest headaches in building a GUI is making sure your app doesn't look like a train wreck when the user starts stretching or resizing the window. This is where **Layout** saves the day. Simply put, a **Layout** is a "rule" for how components are arranged within a `JPanel` or `JFrame`.

Just imagine a **Layout** as an interior designer, each designer has their own principles for arranging the interior of their rooms. Likewise, there are many types of **Layout**, and each type has its own rules or principles for arranging components:

BorderLayout

Divides the area into five main regions: **North**, **South**, **East**, **West**, and **Center**. The Center region automatically takes up whatever remaining space is available.

Documentation: [BorderLayout \(Java SE 17 & JDK 17\)](#)

Declaration:

```
BorderLayout layout = new BorderLayout(20, 20);
```



Example:

```
import javax.swing.*;
import java.awt.*;

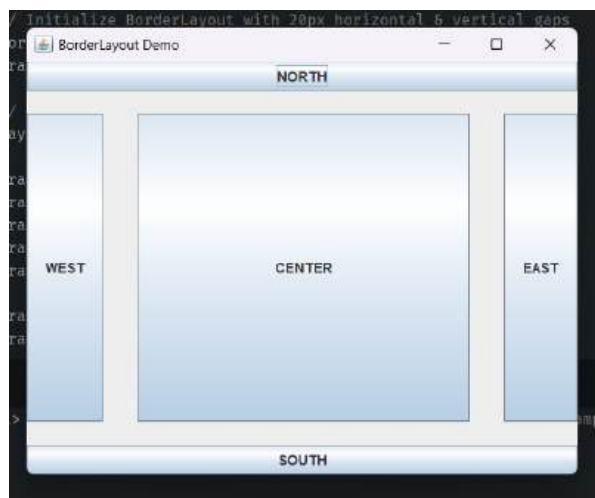
public class BorderLayoutExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("BorderLayout Demo");
        // Initialize BorderLayout with 20px horizontal & vertical gaps
        BorderLayout layout = new BorderLayout(20, 20);
        frame.setLayout(layout);

        // Dynamically change the gap using the setter method
        layout.setHgap(30);

        frame.add(new JButton("NORTH"), BorderLayout.NORTH);
        frame.add(new JButton("SOUTH"), BorderLayout.SOUTH);
        frame.add(new JButton("EAST"), BorderLayout.EAST);
        frame.add(new JButton("WEST"), BorderLayout.WEST);
        frame.add(new JButton("CENTER"), BorderLayout.CENTER);

        frame.setSize(500, 400);
        frame.setVisible(true);
    }
}
```

Output:



FlowLayout

Arranges components in a single line, just like words in a text sentence. Perfect for a collection of buttons that have natural sizes.

Documentation: [FlowLayout \(Java SE 21 & JDK 21\)](#)



Declaration:

```
FlowLayout layout = new FlowLayout(FlowLayout.LEFT, 10, 10);
```

Example:

```
import javax.swing.*;
import java.awt.*;

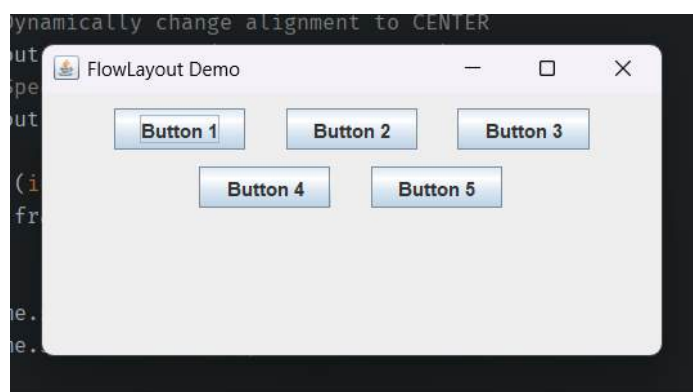
public class FlowLayoutExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("FlowLayout Demo");
        // Create left-aligned layout with 10px horizontal & vertical gaps
        FlowLayout layout = new FlowLayout(FlowLayout.LEFT, 10, 10);
        frame.setLayout(layout);

        // Dynamically change alignment to CENTER
        layout.setAlignment(FlowLayout.CENTER);
        // Specifically widen the horizontal gap
        layout.setHgap(25);

        for (int i = 1; i <= 5; i++) {
            frame.add(new JButton("Button " + i));
        }

        frame.setSize(400, 200);
        frame.setVisible(true);
    }
}
```

Output:



GridLayout

Divides the area into a grid of cells with uniform sizes. Very effective for creating interfaces like calculators or game boards.

Documentation: [GridLayout \(Java SE 22 & JDK 22\)](#)



Declaration:

```
GridLayout layout = new GridLayout(2, 2, 10, 10);
```

Example:

```
import javax.swing.*;
import java.awt.*;

public class GridLayoutExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("GridLayout Demo");
        // Create a 2x2 grid with 10px horizontal & vertical gaps
        GridLayout layout = new GridLayout(2, 2, 10, 10);
        frame.setLayout(layout);

        // Dynamically change row count to 3
        layout.setRows(3);
        // Increase vertical gap between rows
        layout.setVgap(15);

        frame.add(new JButton("1"));
        frame.add(new JButton("2"));
        frame.add(new JButton("3"));
        frame.add(new JButton("4"));
        frame.add(new JButton("5"));
        frame.add(new JButton("6"));

        frame.setSize(300, 300);
        frame.setVisible(true);
    }
}
```



Output:



Actually, there are still other **Layout Managers**, but you can explore those on your own. Here, I'm only giving you the ones that are the easiest and the ones you'll use most often.

2.3.4 Basic Component

So, earlier we talked about `JFrame` (the house), `JPanel` (the rooms), and **Layout** (the interior designer). Now, let's talk about **GUI components** (the furniture). Swing is packed with components, but these four are basically the "bread and butter" for every dev out there:

JLabel

Used to show static text or quick instructions on the screen. Users can't edit this, it's purely there for visual info.

Documentation: [JLabel \(Java SE 21 & JDK 21\)](#)

Declaration:

```
JLabel label = new JLabel();
```



Example:

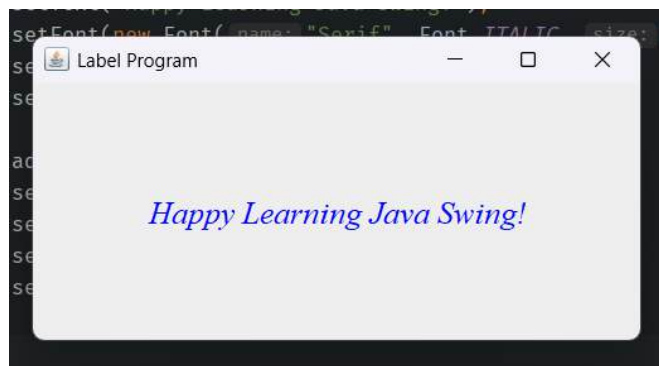
```
import javax.swing.*;
import java.awt.*;

public class LearnLabel {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Label Program");
        JLabel label = new JLabel();

        // Label Configuration
        label.setText("Happy Learning Java Swing!");
        label.setFont(new Font("Serif", Font.ITALIC, 20));
        label.setForeground(Color.BLUE);
        label.setHorizontalAlignment(JLabel.CENTER);

        frame.add(label);
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Output:



JButton

The main action star. Users click this to trigger commands or fire off functions in your app.

Documentation: [JButton \(Java SE 21 & JDK 21\)](#)

Declaration:

```
JButton button = new JButton("Click Me");
```



Example:

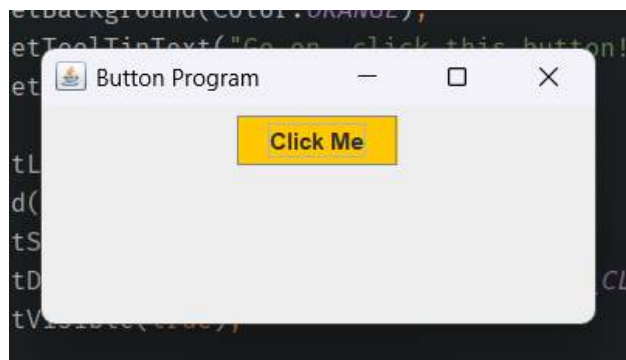
```
import javax.swing.*;
import java.awt.*;

public class LearnButton {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Program");
        JButton button = new JButton("Click Me");

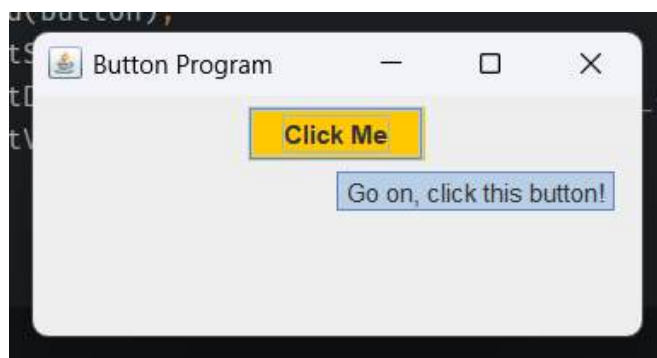
        // Button Configuration
        button.setBackground(Color.ORANGE);
        button.setToolTipText("Go on, click this button!");
        button.setEnabled(true);

        frame.setLayout(new FlowLayout());
        frame.add(button);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

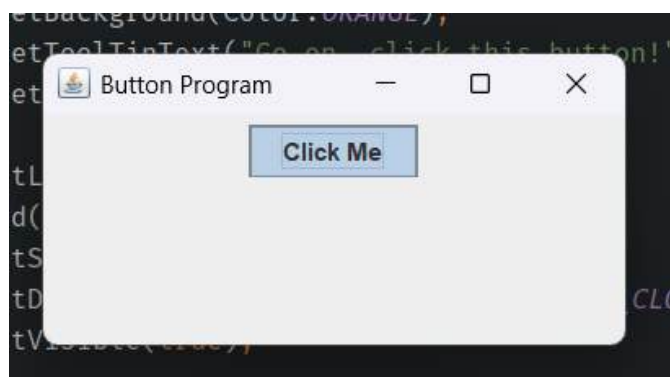
Output:



When You hover cursor to the button:



When You click the button:



JTextField

A simple one-line box where users can type in input, like usernames or numbers for calculations.

Documentation: [JTextField \(Java SE 21 & JDK 21\)](#)

Declaration:

```
JTextField inputField = new JTextField();
```

Example:

```
import javax.swing.*;
import java.awt.*;

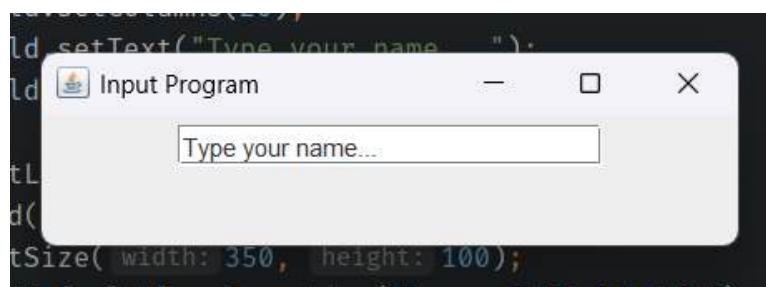
public class LearnInput {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Input Program");
        JTextField inputField = new JTextField();

        // Input Field Configuration
        inputField.setColumns(20);
        inputField.setText("Type your name...");
        inputField.setEditable(true);

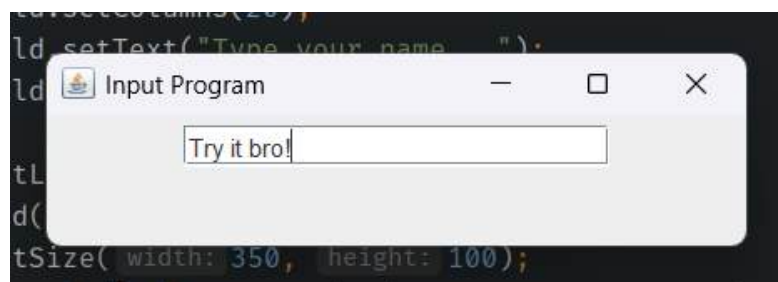
        frame.setLayout(new FlowLayout());
        frame.add(inputField);
        frame.setSize(350, 100);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```



Output (default):



When You type something:



JOptionPane

A special component for quick pop-up messages. Use it for warnings, confirmations, or just to show some quick info without the hassle of building a complicated new `JFrame` from scratch.

You don't need to create an object to call `JOptionPane`. Just import the library, then simply type `JOptionPane`, and you can use any method available in `JOptionPane`.

Documentation: [JOptionPane \(Java SE 21 & JDK 21\)](#)

Example:

```
import javax.swing.JOptionPane;

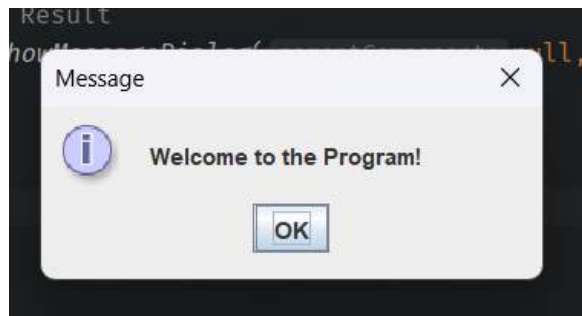
public class LearnPopUp {
    public static void main(String[] args) {
        // Show Information Message
        JOptionPane.showMessageDialog(null, "Welcome to the Program!");

        // Get Name Input
        String name = JOptionPane.showInputDialog("What is your name?");

        // Show Input Result
        JOptionPane.showMessageDialog(null, "Hello, " + name + "!");
    }
}
```



Output:



JTable

Used to display data in a table format with rows and columns. This component supports cell editing, sorting, and custom rendering.

Documentation: [JLabel \(Java SE 21 & JDK 21\)](#)

Declaration:

```
// Table data (rows and columns)
String[][] data = {
    {"Budi", "21"},
    {"Ani", "22"},
    {"Siti", "20"},
    {"Andi", "23"}
};

// Column names
String[] columnNames = {"Name", "Age"};

// Create the JTable
JTable table = new JTable(data, columnNames);
```



Example:

```
import javax.swing.JFrame;
import javax.swing.JScrollPane;
import javax.swing.JTable;

public class JTableExample {
    public static void main(String[] args) {
        // Table data (rows and columns)
        String[][] data = {
            {"Budi", "21"},
            {"Ani", "22"},
            {"Siti", "20"},
            {"Andi", "23"}
        };

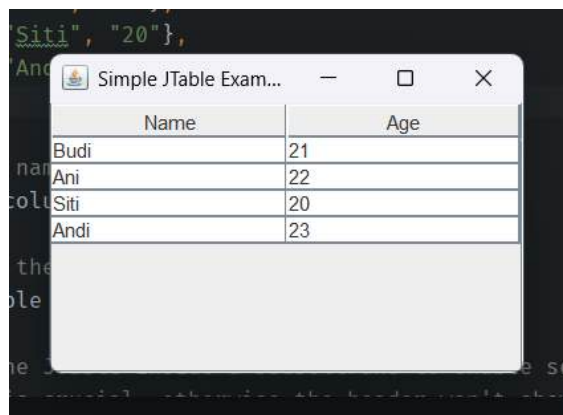
        // Column names
        String[] columnNames = {"Name", "Age"};

        // Create the JTable
        JTable table = new JTable(data, columnNames);

        // Wrap the JTable inside a JScrollPane to enable scrolling
        // (This is crucial, otherwise the header won't show up!)
        JScrollPane scrollPane = new JScrollPane(table);

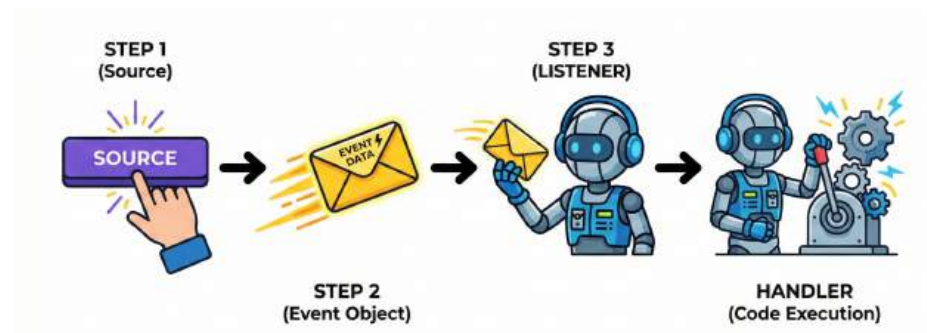
        // Create a JFrame to display the table
        JFrame frame = new JFrame("Simple JTable Example");
        frame.add(scrollPane);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:



2.3.5 Event Handling in Swing

So, earlier we talked about the appearance (`JFrame`, `JPanel`, **Layout**, **Components**), now we're talking about the **ACTION**. **Event Handling**, simply put, is the concept that gives actions to each GUI component (like button). Without **Event Handling**, all buttons wouldn't work when clicked. Therefore, to make a button functional, we apply **Event Handling**. With **Event Handling**, you can do things like "click! -> Bang!"-thing.



Do you still remember these three concepts?

- **Event Source (The Trigger)**
- **Event Object (The Messenger)**
- **Event Listener (The Responder)**

We discussed them back in Sub-Chapter 1 about GUI (at the very beginning of this module). We will apply these three concepts in Swing to perform Event Handling, which means "bringing button functions to life".

Documentation: [Lesson: Writing Event Listeners \(The Java™ Tutorials > Creating a GUI With Swing\)](#)

How to Make Event Handling

To give a component (like a `JButton`) some interactive superpowers, you need to go through a systematic logic assembly process. Here are the three deep-dive steps along with the code:

- **Step 1: Create the Listener Object** First up, you need to set up the listener that's going to be on standby. In Java, we use the `ActionListener` interface. The coolest and most concise way for beginners to do this is by making an **Anonymous Class**.



```
// Create the click listener object
ActionListener listener = new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        // Code here isn't executed yet, just prepared.
    }
};
```

Here, we are basically signing a contract. We promise that if something goes down, the code inside the `actionPerformed` block is the one responsible for handling it.

- **Step 2:** Write the Logic inside `actionPerformed` Now that the container is ready, we need to fill in the "work instructions" inside the `actionPerformed` method. This method gets a parameter called `ActionEvent e`, which carries all the info about that specific click.

```
@Override
public void actionPerformed(ActionEvent e) {
    // Give real instructions
    System.out.println("The button was just clicked!");
    JOptionPane.showMessageDialog(null, "Logic is Running!");
}
```

Whatever you write in here, whether it's math calculations, grabbing text from an input field, or popping up a dialog, will run exactly when the trigger (event) happens.

- **Step 3:** Register to the Event Source Even a great listener won't know *what* to listen to if it isn't attached to the source. We have to use the `addActionListener` method that belongs to the source component (like the button).

```
JButton actionButton = new JButton("Click Here");
// Final step: Attach the listener to the button
actionButton.addActionListener(listener);
```

The line `addActionListener(listener)` is the glue. Without this line, even if you've written the most advanced listener in the world, the button will stay totally silent when clicked because there's no "cable" connecting the button's action to your program's logic.



Here is a summary of those three steps combined into a single piece of code you can try out right away:

```
import javax.swing.*;
import java.awt.event.*;

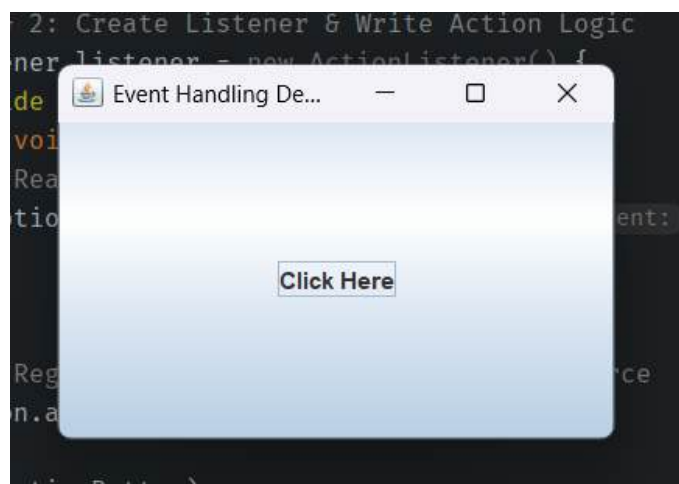
public class EventImplementationSteps {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Event Handling Demo");
        JButton actionButton = new JButton("Click Here");

        // Step 1 & 2: Create Listener & Write Action Logic
        ActionListener listener = new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Real instructions to be executed
                JOptionPane.showMessageDialog(null, "Button Responded Successfully!");
            }
        };

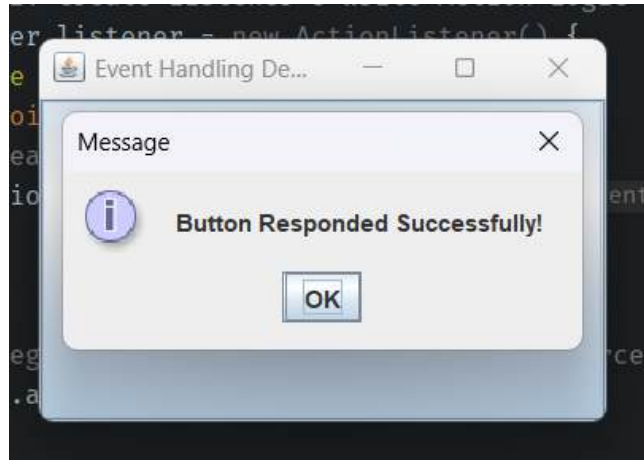
        // Step 3: Register the listener to the event source
        actionButton.addActionListener(listener);

        frame.add(actionButton);
        frame.setSize(300, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }
}
```

Output:



When button clicked:



How is it? Fun, right? It feels great when the button doesn't ignore you and actually responds. So, don't just read other people's chats, kidd :)



3. JavaFX

3.1 Why Should I Understand This?

Earlier we learned Swing, now we're moving on to JavaFX. Swing is old-fashioned, it only lets us create simple interfaces, **not as beautiful as your smile...** I mean, as beautiful as modern applications today. If you really want to build a trendy, up-to-date GUI, you need to learn **JavaFX**.

Next, one of the trickiest things for beginners when building GUIs in Swing is that program logic gets mixed with design code. For example, if you want to change the color of a button, you have to tinker with complicated lines of Java code. Well... JavaFX comes with the philosophy of "**Separation of Concerns**". It allows us to design the application visually or with code similar to building websites (FXML, which is like HTML), while keeping the program logic clean in a separate Java file. In other words, you can tweak the GUI **without touching** the Java logic, and vice versa. Awesome!

Beyond appearance, JavaFX is also lightweight! It's lightweight because it directly utilizes the GPU, unlike its predecessor which relied only on the CPU. That's why applications built with JavaFX run **smoothly and fast**.

3.2 Formal Definition

JavaFX is a software platform and a set of graphics libraries used to build feature-rich desktop apps (Rich Client Applications) and interactive internet applications. It was designed to be the **modern successor to Java Swing**. It comes packed with full support for multimedia, vector graphics, cool animations, and hardware-based processing.

The main structure of JavaFX is built on the **Scene Graph** concept. Think of the Scene Graph as a hierarchical "tree" of data that manages all the visual elements in your app's interface. Every element inside this graph is called a **Node**. A Node can be something simple like a button, or something complex like 3D graphics or a video player.

Here are some of the key features that define JavaFX:

- **FXML:** An XML-based language used to build your UI. It's super handy because it lets you separate the visual design from the actual code logic.
- **CSS Styling:** You can tweak the look and feel of your app using CSS (Cascading Style Sheets)—just like you would when designing a website.
- **Hardware Acceleration:** It uses your computer's GPU to render the interface, which guarantees high performance and smooth visuals.
- **WebView:** A component that lets you embed web content (HTML5) right inside your JavaFX application.



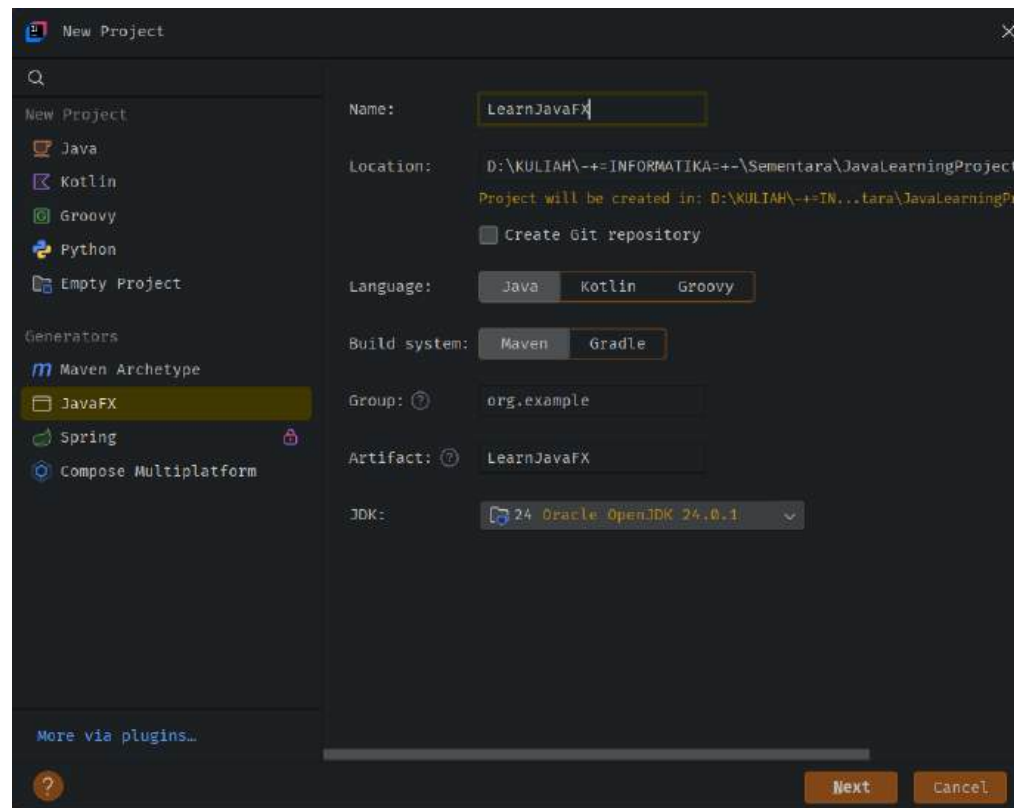
3.3 Concepts Elaboration

3.3.1 Instalation

Well, in Swing, you don't need to create a special new project, you can directly use an existing program and simply import the built-in JDK library, namely `javax.swing`. But in **JavaFX**, you have to install the library manually, because **JavaFX** is no longer bundled with the JDK.

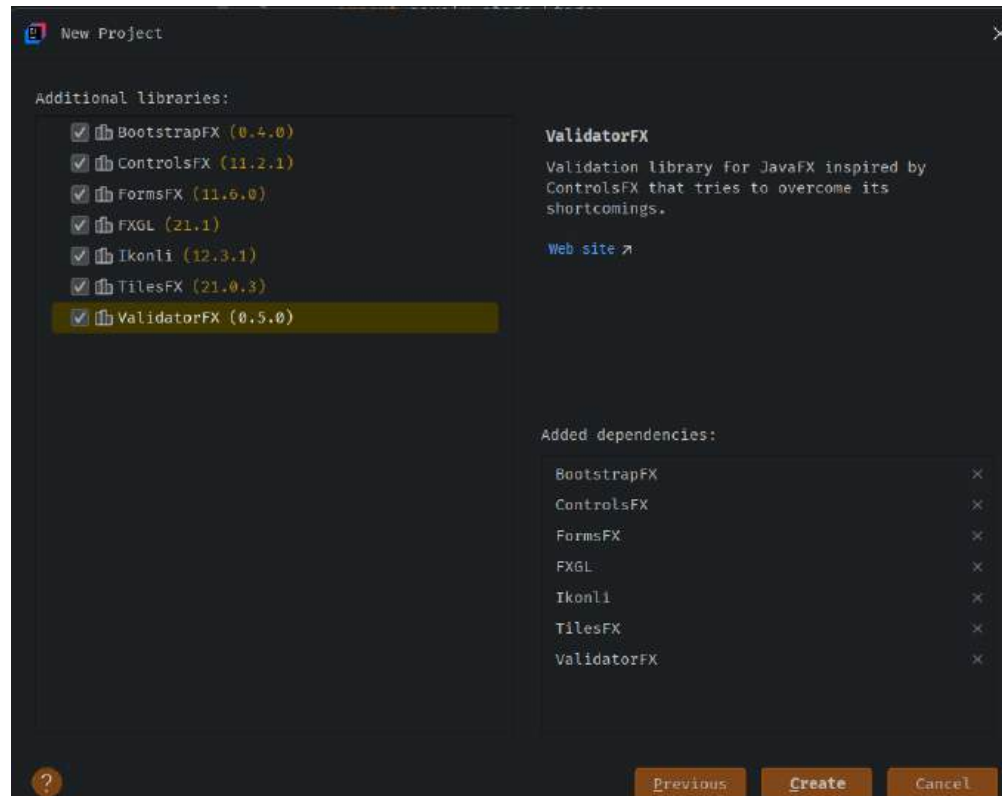
So, the steps are a bit more complicated, please follow along:

- **Step 1:** First, go to the “**New Project**” menu, then look under the Generators section, you'll see many options. Now, click on the “**JavaFX**” option. The project **name** and **location** is up to you, just make sure the **language** is set to Java since we're learning Java. For the **Build System**, choose Maven. Select **JDK version 24**. Click **NEXT**.

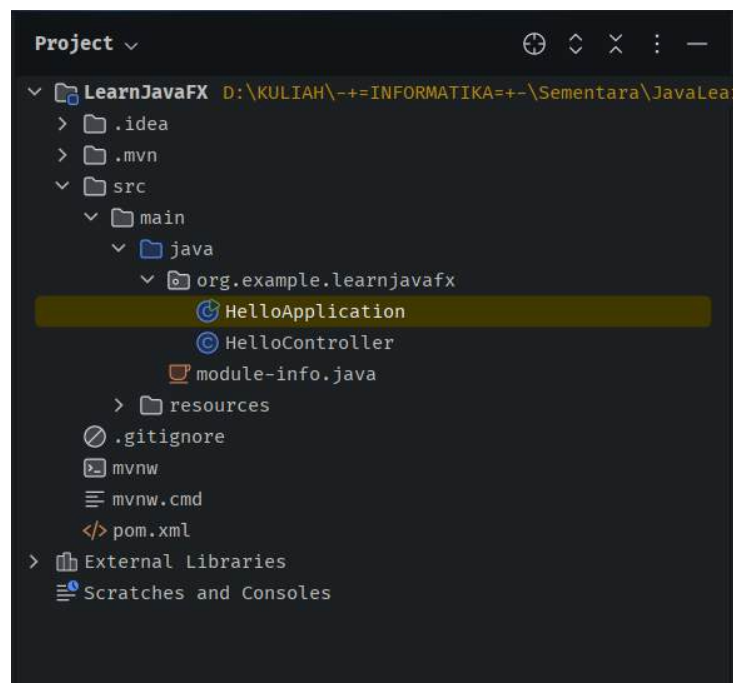


- **Step 2:** After clicking next, you'll be taken to the next page. Don't worry about it for now, just check everything. Then click “**Create**”.

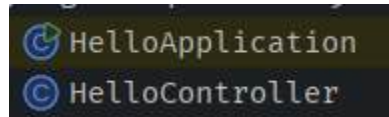




- **Step 3:** Now, with the Maven build system, the project structure is a little different. If you're confused about where to find the code, just open **src -> main -> java -> org.example.project_name**.



- **Step 4:** Here you will find two default files. The first file, "HelloApplication.java", contains two methods: `start()` (the place for GUI code) and `main()` (the entry point when the code is executed).



- **Step 5:** For now, you'll get a taste of JavaFX-style GUI code inside the "HelloApplication.java" file, specifically in the `start()` method.

```
package org.example.learnjavafx;

import javafx.application.Application;
import javafx.stage.Stage;

import java.io.IOException;

public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        //GUI CODE HERE
    }

    public static void main(String[] args) {
        launch();
    }
}
```



3.3.1 Stage



In Swing we have `JFrame`, while in JavaFX there is something called `Stage`. The concept is very similar to `JFrame`, which serves as the main container. We can use the same analogy as well, where `Stage` is like a house that will later contain many rooms and furniture.

Documentation: [Stage \(JavaFX 21\)](#)

- **Primary Stage:** When you first create a JavaFX project, IntelliJ automatically generates one main `Stage` for you, usually called `PrimaryStage`. Here is an example of the `PrimaryStage`:

```
package org.example.learnjavafx;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;

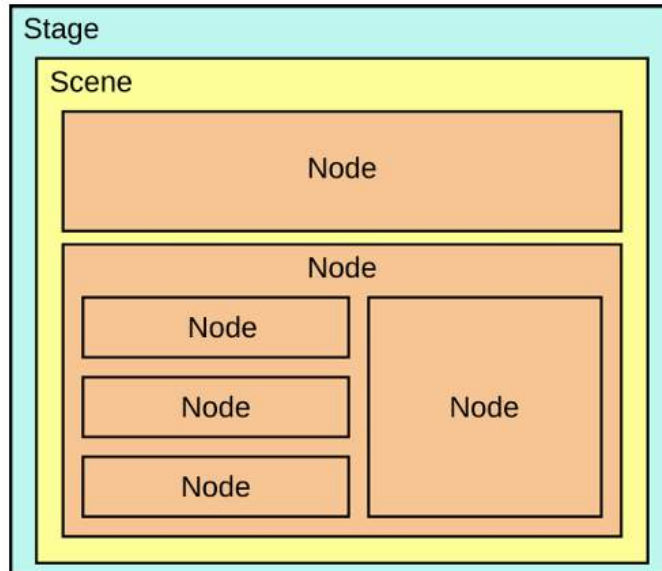
public class HelloApplication extends Application {
    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader fxmlLoader = new FXMLLoader(HelloApplication.class.getResource("hello-view.fxml"));
        Scene scene = new Scene(fxmlLoader.load(), 320, 240);
        stage.setTitle("Hello!");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```



- **Manual Stage Creation:** You can also manually create a new Stage from scratch if you want to display a new window in your application.

Container Hierarchy in JavaFX (Stage-Scene-Node)



In JavaFX, there is a **hierarchy** you need to understand so you don't get confused:

- **Stage:** The highest-level container (Windows/Mac window). It holds all the GUI content you create and later see.
- **Scene:** The content inside that window. Like a play in a theater, one Stage can switch between different Scenes (for example: from a Login Scene to a Dashboard Scene).
- **Nodes:** The visual elements inside a Scene (buttons, text, images).



3.3.2 Scene



If *Stage* is the house, then *Scene* is the contents or the "atmosphere" inside that house. You can imagine *Scene* as a scene that changes on a theater stage. In real application use, *Scene* represents the application's pages, such as the dashboard page, profile page, and others that can switch within the same window.

Here are some facts about *Scene*:

- You can write code for the Login view and the Main view in two different *Scene* objects so the code doesn't pile up in one place.
- To switch pages, you just call `stage.setScene(newScene)`. The *Stage* remains the same, the window is the same, but the page inside that window can change.
- CSS files (used to style the appearance) are usually attached at the *Scene* level, not at the *Stage* level.

Documentation: [Scene \(JavaFX 21\)](#)

Declaration:

To create a *Scene*, we need at least two things: a **Root Node** (the main layout) and the **dimensions** (width & height). We will discuss **Layout** in the next section. Stay tuned.

```
// root is a layout like VBox, HBox, or GridPane
Scene scene = new Scene(root, 400, 300);
```



Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class HelloScene extends Application {
    @Override
    public void start(Stage stage) {
        // 1. Create a Node (Content inside the Scene)
        Label label = new Label("This is content inside the Scene!");

        // 2. Create the Root Node (Layout)
        StackPane root = new StackPane();
        root.getChildren().add(label);

        // 3. Create the Scene and insert the root into it
        // Size: width 400px, height 200px
        Scene scene = new Scene(root, 400, 200);

        // 4. Set the Scene onto the Stage
        stage.setTitle("Learning JavaFX Scene");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch();
    }
}
```

Output:



3.3.3 Layout

After understanding `Stage` and `Scene`, we will now learn how to arrange the position of components inside a `Scene` so that they look neat and responsive. In JavaFX, we use **Layout** to define the rules for positioning elements.

Now, this **Layout** is what you will use when creating a `Scene`. An interesting fact about layouts in JavaFX is that you can perform **nested layouts**, which means placing a layout inside another layout as many times as you want.

Some commonly used Layouts include:

- **HBox & VBox**: Arrange elements horizontally or vertically.
- **GridPane**: Arrange elements in rows and columns (like a table).
- **BorderPane**: Divide the area into five regions (Top, Bottom, Left, Right, Center).
- **StackPane**: Stack elements on top of each other.

HBox & VBox

`HBox` (**Horizontal Box**) and `VBox` (**Vertical Box**) are the most basic containers that arrange elements in a single linear line. `HBox` organizes all components horizontally from left to right, while `VBox` arranges them vertically from top to bottom. These containers are very efficient for creating interfaces that are simple yet well-structured.

Documentation: [VBox \(JavaFX 21\)](#), [HBox \(JavaFX 21\)](#)

Declaration:

```
VBox root = new VBox();
```

```
HBox hbox = new HBox();
```

VBox Example:



```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class VBoxExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create VBox (vertical container)
        VBox root = new VBox();

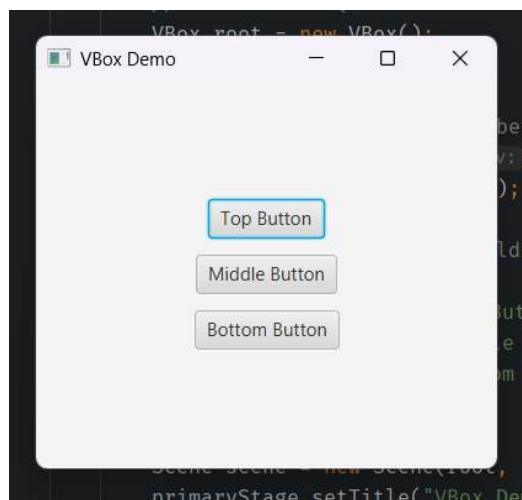
        // Use important methods
        root.setSpacing(10); // Gap between buttons
        root.setPadding(new Insets(20)); // Gap from window edges
        root.setAlignment(Pos.CENTER); // All buttons in the middle

        // Add components to the children list
        root.getChildren().addAll(
            new Button("Top Button"),
            new Button("Middle Button"),
            new Button("Bottom Button")
        );

        Scene scene = new Scene(root, 300, 250);
        primaryStage.setTitle("VBox Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

VBox Output:



HBox Example:

```
package org.example.demo.TEST;

import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.HBox;
import javafx.stage.Stage;

public class HBoxExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        // Create HBox (horizontal container)
        HBox root = new HBox();

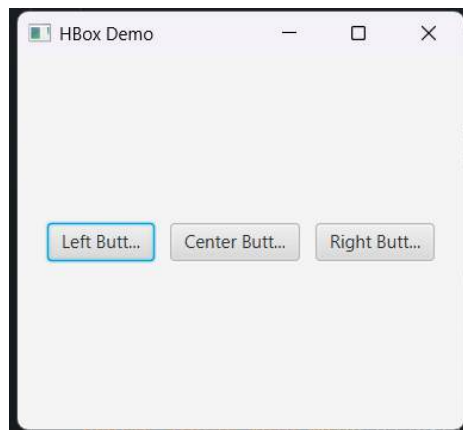
        // Use important methods
        root.setSpacing(10); // Gap between buttons
        root.setPadding(new Insets(20)); // Gap from window edges
        root.setAlignment(Pos.CENTER); // All buttons in the middle

        // Add components to the children list
        root.getChildren().addAll(
            new Button("Left Button"),
            new Button("Center Button"),
            new Button("Right Button")
        );

        Scene scene = new Scene(root, 300, 250);
        primaryStage.setTitle("HBox Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

HBox Output:



GridPane

If your application requires a layout resembling a **registration** form or a **chessboard**, then `GridPane` is the right choice. This container works with a row-and-column system. Each component is placed at a **specific index**, for example at row 0 and column 1.

Documentation: [GridPane \(JavaFX 21\)](#)

Declaration:

```
GridPane grid = new GridPane();
```

Example:

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class GridExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        GridPane grid = new GridPane();
        grid.setPadding(new Insets(20));

        // Set gap between cells
        grid.setHgap(10);
        grid.setVgap(10);

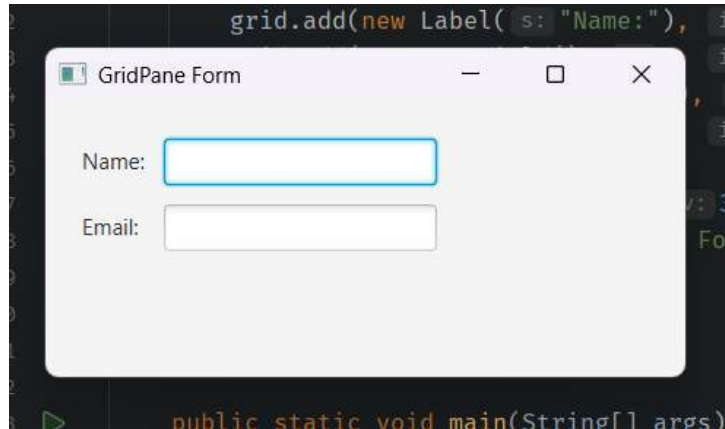
        // Add components: add(node, column, row)
        grid.add(new Label("Name:"), 0, 0);
        grid.add(new TextField(), 1, 0);
        grid.add(new Label("Email:"), 0, 1);
        grid.add(new TextField(), 1, 1);

        Scene scene = new Scene(grid, 350, 150);
        primaryStage.setTitle("GridPane Form");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```



Output:



BorderPane

`BorderPane` is a very popular container because it provides a ready-made framework for typical desktop applications. It divides the window area into five main regions: **Top**, **Bottom**, **Left**, **Right**, and **Center**.

Documentation: [BorderPane \(JavaFX 21\)](#)

Declaration:

```
BorderPane layout = new BorderPane();
```

Example:



```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.BorderPane;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

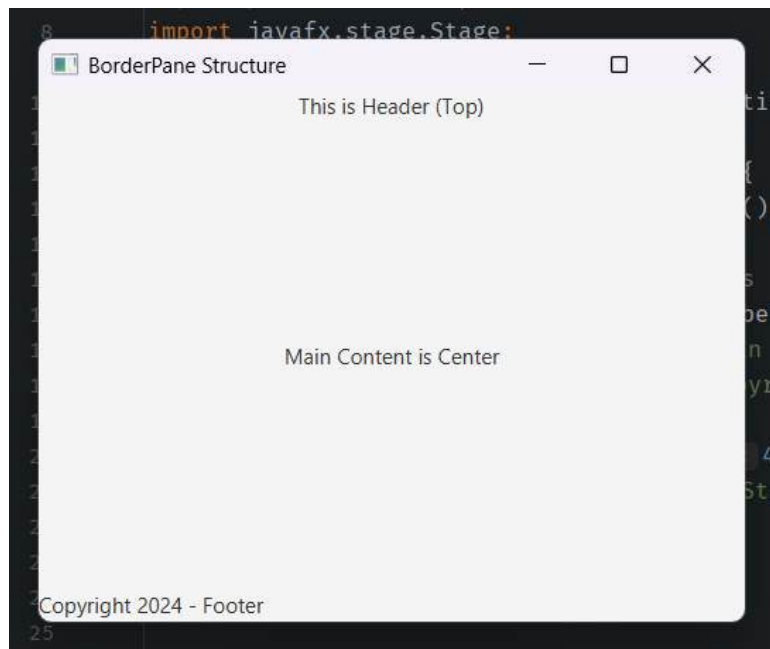
public class BorderExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        BorderPane layout = new BorderPane();

        // Set components in specific areas
        layout.setTop(new StackPane(new Label("This is Header (Top)")));
        layout.setCenter(new Label("Main Content is Center"));
        layout.setBottom(new Label("Copyright 2024 - Footer"));

        Scene scene = new Scene(layout, 400, 300);
        primaryStage.setTitle("BorderPane Structure");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



AnchorPane

`AnchorPane` lets you hook or anchor elements to the edges of the container. For example, you can set a button to always be 10 pixels from the right edge. When the user resizes the window, the button stays locked to that 10-pixel distance from the new edge.

Documentation: [AnchorPane \(JavaFX 21\)](#)

Declaration:

```
AnchorPane anchor = new AnchorPane();
```

Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.AnchorPane;
import javafx.stage.Stage;

public class AnchorExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        AnchorPane anchor = new AnchorPane();
        Button btnSave = new Button("Save");

        // Lock button position to bottom-right corner
        AnchorPane.setBottomAnchor(btnSave, 15.0);
        AnchorPane.setRightAnchor(btnSave, 15.0);

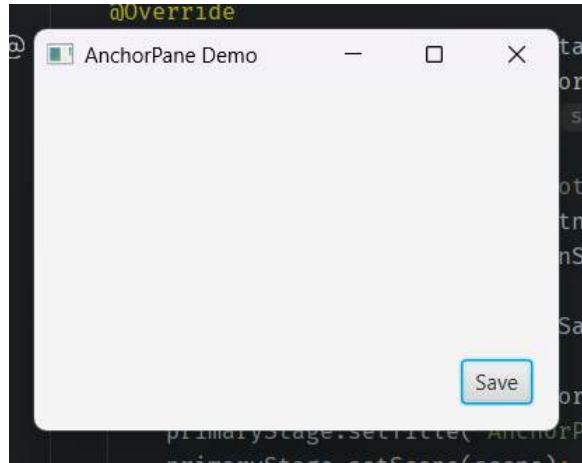
        anchor.getChildren().add(btnSave);

        Scene scene = new Scene(anchor, 300, 200);
        primaryStage.setTitle("AnchorPane Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:





StackPane

`StackPane` stacks components on top of each other in layers. The first element added becomes the bottom layer, and each subsequent element is stacked on top. This is perfect for creating overlays or layered visual effects.

Documentation: [StackPane \(JavaFX 21\)](#)

Declaration:

```
StackPane stack = new StackPane();
```

Example:



```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

public class StackExample extends Application {
    @Override
    public void start(Stage primaryStage) {
        StackPane stack = new StackPane();

        // Create a rectangle as background
        Rectangle bg = new Rectangle(200, 100);
        bg.setFill(Color.LIGHTBLUE);

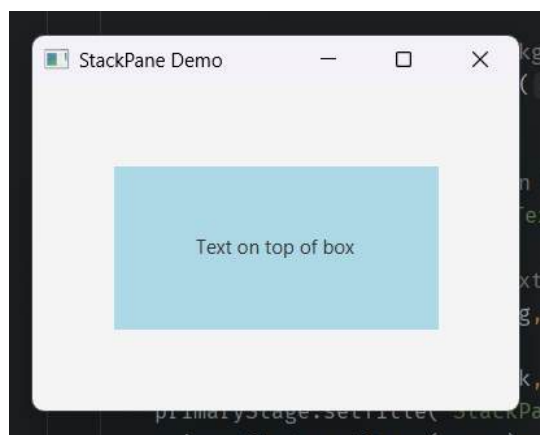
        // Create a label to stack on top
        Label text = new Label("Text on top of box");

        // The last object added (text) will be in front
        stack.getChildren().addAll(bg, text);

        Scene scene = new Scene(stack, 300, 200);
        primaryStage.setTitle("StackPane Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



3.3.4 Components

After the house framework (layout) is ready, it's time to furnish the house with furniture, namely **UI Controls**. These are the elements that can be seen and directly interacted with by the user. JavaFX provides a rich collection of controls that are **highly customizable**. In JavaFX, components are called "**Nodes**".

Let's look at the most essential ones:

Label

The `Label` is the simplest component. Its job is to display static text or information that the user can't edit directly. Usually, you use Labels for titles, form instructions, or just descriptive tags next to input fields.

Documentation: [Label \(JavaFX 12\)](#)

Declaration:

```
Label label = new Label("Hello, This is JavaFX!");
```

Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.stage.Stage;

public class MainLabel extends Application {
    @Override
    public void start(Stage stage) {
        // Create label with blue text and large font
        Label label = new Label("Hello, This is JavaFX!");
        label.setTextFill(Color.DARKBLUE);
        label.setFont(new Font("Arial", 24));

        StackPane root = new StackPane(label);
        stage.setScene(new Scene(root, 300, 150));
        stage.setTitle("Label Example");
        stage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

Output:





Button

The `Button` is the main interactive component used to trigger an action or program logic when clicked. In JavaFX, buttons are super flexible, they can hold text, images/icons, or a mix of both to look cooler.

Documentation: [Button \(JavaFX 21\)](#)

Declaration:

```
Button btn = new Button("Click Me!");
```

Example:

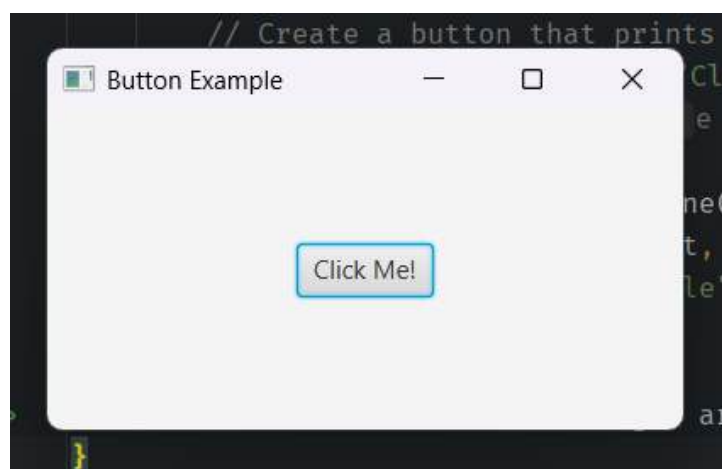
```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MainButton extends Application {
    @Override
    public void start(Stage stage) {
        // Create a button that prints a message when clicked
        Button btn = new Button("Click Me!");
        btn.setOnAction(e -> System.out.println("Button clicked!"));

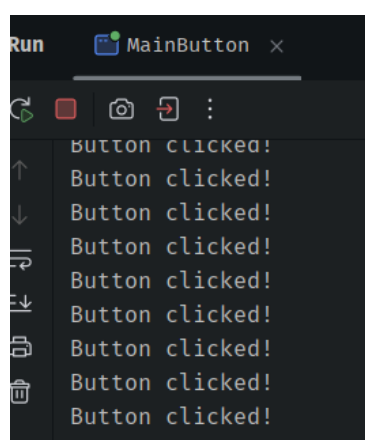
        StackPane root = new StackPane(btn);
        stage.setScene(new Scene(root, 300, 150));
        stage.setTitle("Button Example");
        stage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

Output:





Console Output:



TextField

`TextField` and `PasswordField` work in tandem to accept single-line text input from users. The main difference is security, `TextField` shows text as-is, while `PasswordField` masks typed characters to protect sensitive data like passwords.

Documentation: [TextField \(JavaFX 21\)](#)

Declaration:

```
TextField nameField = new TextField();
```

Example:



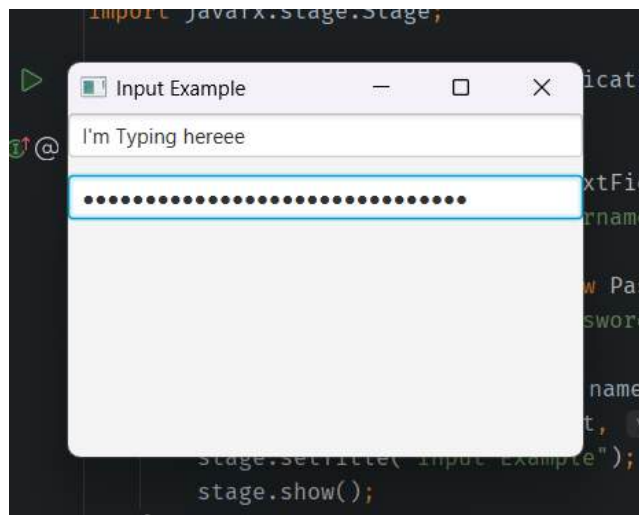
```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class MainFields extends Application {
    @Override
    public void start(Stage stage) {
        TextField nameField = new TextField();
        nameField.setPromptText("Username");

        PasswordField passField = new PasswordField();
        passField.setPromptText("Password");

        VBox root = new VBox(10, nameField, passField);
        stage.setScene(new Scene(root, 300, 200));
        stage.setTitle("Input Example");
        stage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

Output:



ListView & TableView

`ListView` and `TableView` are designed to present large collections of data in an organized way. **ListView** shows a simple single-column list, while `TableView` is much more powerful, capable of displaying data in a grid with sortable columns.

Documentation: [ListView \(JavaFX 21\)](#), [TableView \(JavaFX 21\)](#)

Declaration:




```
ListView<String> list = new ListView<>();
```

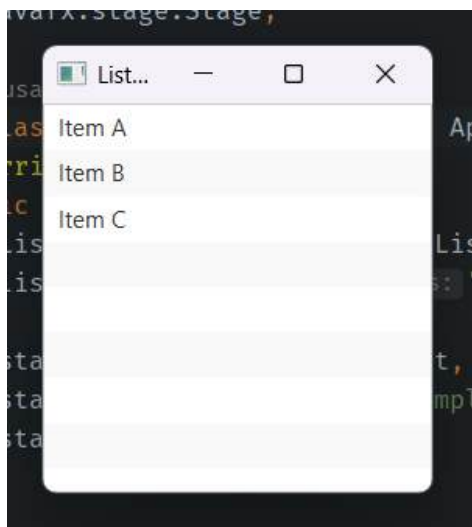
Example:

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.ListView;
import javafx.stage.Stage;

public class MainListView extends Application {
    @Override
    public void start(Stage stage) {
        ListView<String> list = new ListView<>();
        list.getItems().addAll("Item A", "Item B", "Item C");

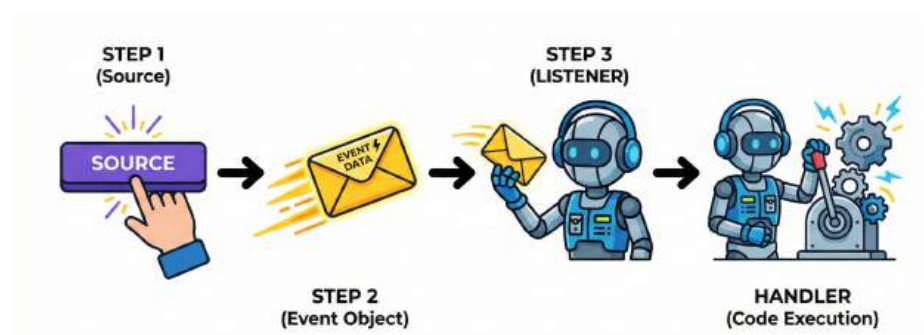
        stage.setScene(new Scene(list, 200, 200));
        stage.setTitle("ListView Example");
        stage.show();
    }
    public static void main(String[] args) { launch(args); }
}
```

Output:



3.3.5 Event Handling

Now, let's return to the discussion of **Event Handling**, but this time in the context of **JavaFX**, not **Swing**. The concept of **Event Handling** is exactly the same as in Swing, which is to make GUI components react when we click them.



Event Handling in JavaFX is also based on three concepts that we have discussed many times:

- **Event Source** (The Trigger)
- **Event Object** (The Messenger)
- **Event Listener** (The Responder)

We discussed them back in Sub-Chapter 1 about GUI (at the very beginning of this module). We will apply these three concepts in Swing to perform Event Handling, which means “bringing component functions to life”.

How to Make Event Handling

To give a component (like a `Button`) some interactive superpowers, you need to go through a systematic logic assembly process. Here are the three deep-dive steps along with the code:

- **Step 1: Create the Handler Object** In JavaFX, we don't use `ActionListener`, but instead, we use the `EventHandler` interface. While you can use an Anonymous Class, JavaFX developers almost always use **Lambda Expressions** to keep the code short and sweet.

```

// Using Lambda Expression (The Modern Way)
EventHandler<ActionEvent> handler = e -> {
    // Code here isn't executed yet, just prepared.
};
  
```



- **Step 2: Write the Logic inside the Handler** Now, we fill the "work instructions" inside the event handler. This block will receive an `ActionEvent` object which contains all the details about the interaction.

```
- EventHandler<ActionEvent> handler = e -> {
    // Give real Instructions
    System.out.println("JavaFX Button Clicked!");
    // Pop up a message (JavaFX style)
    Alert alert = new Alert(Alert.AlertType.INFORMATION);
    alert.setContentText("Logic is running in JavaFX!");
    alert.show();
};
```

- **Step 3: Register to the Event Source** Finally, we connect the logic to the component. In JavaFX, instead of `addActionListener`, we use the `setOnAction` method.

```
Button btn = new Button("Click Me");
// Final step: Attach the handler to the button
btn.setOnAction(handler);
```

Here is a summary of those three steps combined into a single piece of code. You can put this inside your `start()` method:



```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class JavaFXEventDemo extends Application {
    @Override
    public void start(Stage stage) {
        Button actionButton = new Button("Click Here");

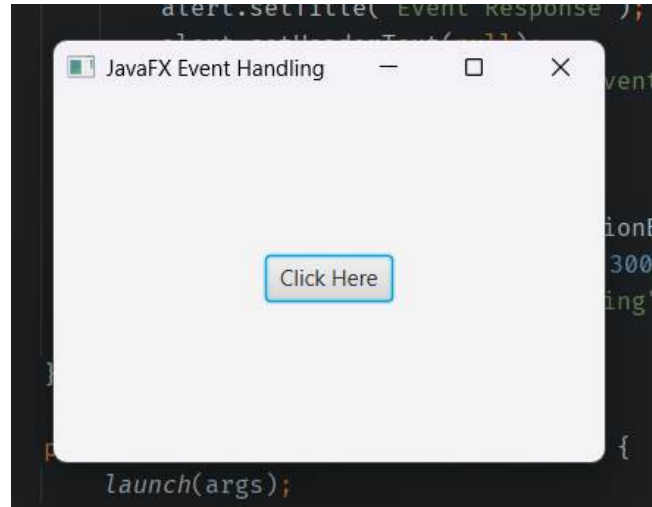
        // Step 1, 2, & 3 combined:
        actionButton.setOnAction(e -> {
            Alert alert = new Alert(Alert.AlertType.INFORMATION);
            alert.setTitle("Event Response");
            alert.setHeaderText(null);
            alert.setContentText("JavaFX Event Handled Successfully!");
            alert.showAndWait();
        });

        StackPane root = new StackPane(actionButton);
        stage.setScene(new Scene(root, 300, 200));
        stage.setTitle("JavaFX Event Handling");
        stage.show();
    }

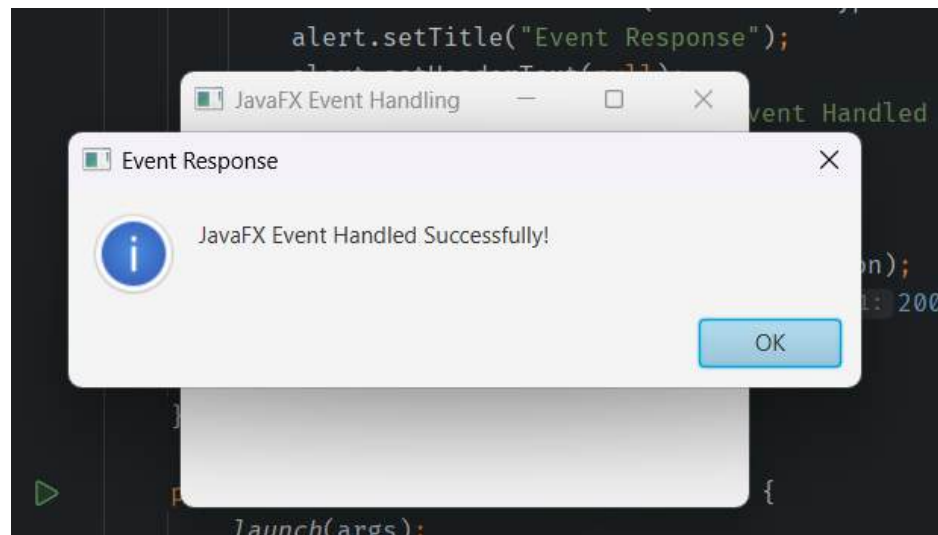
    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:





When You click the button:



EXERCISE

Here, we won't need the last exercise, we will start over from scratch. Let's make a simple game called Number Guesser. Here's what you need to do:

1. Use library called `Random` to randomize a number between 1 - 100

```
1 import java.util.Random;
2
3 Random random = new Random();
4 targetNumber = random.nextInt(100) + 1;
```

2. When the program is run, the user can input any number between 1 - 100
3. If the user guesses correctly, the game will end, and the program asks if the user wants to play again
4. If the user's guess is too high compared to the target number, there's text that tells the user the guess number is too high
5. If the user's guess too low, the text will say it's too low

Here's some example:





Here's what you should understand:

1. What's VBox?
2. What's the difference between VBox and HBox?
3. What's CSS styling?
4. What do you call the box that the user can input text in?
5. How to get input from a text field?



ASSIGNMENT

The Mayor felt the CLI (Black Screen) system was too outdated and difficult for administrative staff to understand and use. He could replace the CLI system with a desktop application featuring buttons, input forms, and data tables.

The main task: create a new class called `MainApp.java` that extends `Application`. The CLI version of `Main.java` will be replaced with a GUI. Here's what you need to do:

1. The top must be a header off the app
2. The center is an area where you can add text fields or a table, and a button to save or delete

```

1 // 1. Setup TableView
2     TableView<Building> table = new TableView<>();
3     table.setItems(buildingData);
4
5     // Kolom 1: Name
6     TableColumn<Building, String> nameCol = new TableColumn<>("Name");
7     nameCol.setCellValueFactory(new PropertyValueFactory<>("name"));
8
9     // Kolom 2: Address
10    TableColumn<Building, String> addressCol = new TableColumn<>("Address");
11    addressCol.setCellValueFactory(new PropertyValueFactory<>("address"));
12
13    // Kolom 3: Status
14    TableColumn<Building, BuildingStatus> statusCol = new TableColumn<>("Status");
15    statusCol.setCellValueFactory(new PropertyValueFactory<>("status"));
16
17    // Kolom 4: Building Type
18    TableColumn<Building, String> typeCol = new TableColumn<>("Type");
19    typeCol.setCellValueFactory(cellData ->
20        new javafx.beans.property.SimpleStringProperty(
21            cellData.getValue().getClass().getSimpleName() // Get Name from
22            Hospital/Market
23        )
24    );
  
```

3. Use text field to input the name, address, and number of floors
4. Use a combo box to choose Building status (You can use Enum for combo box)



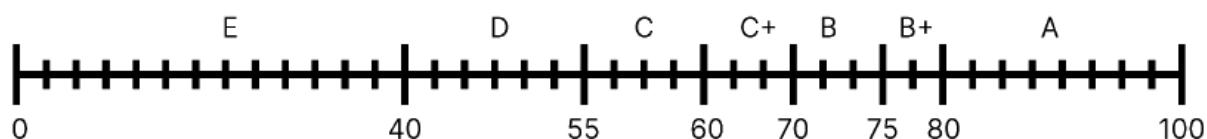
- 



ASSESSMENT

ASSESSMENT ASPECT	POINT
Codelab	20%
Assignment	80%
Code Running Smoothly	30%
2 New Features	10%
3 New Features	10%
4 New Features	10%
Answer Question from the Module	20%
Total	100%

Percentage Assessment



- A** = (81 - 100) → **You're The Chosen One**
- B+** = (75 - 80) → **Great!**
- B** = (70 - 74) → **Good!**
- 67** = (67) → **Square root of 4489**
- C+** = (60 - 69) → **Keep it up!**
- C** = (55 - 59) → **Almost**
- D** = (41 - 54) → **Study Some More!**
- E** = (0 - 40) →



Closing Statement?

Glow up alert! 🌟 Your program isn't just a depressing black-and-white block of text anymore — it's finally got some color and buttons!!

Sure, the logic behind the scenes might be a chaotic mess, but hey... at least it looks pretty, right?



Finally, you've completed all the modules. Finallyyy, finallyyyyy! You should be proud of yourselves.



But don't celebrate just yet, because the FINAL EXAM is still waiting for you.

